helpers

```
SPD_MAX_DIST = 8          # distances above this share one bias (SPDAttention.max_dist)
DENSE_LIMIT = 1024        # graphs this small are handled with dense linear algebra


def laplacian_eigvecs(edge_index, num_nodes, k=LAP_K):
    """Rows of the k eigenvectors of L = I - D^-1/2 A D^-1/2 with the smallest
    non-trivial eigenvalues, scaled by sqrt(n) so each channel has unit RMS.
    Padded with zero columns when the graph has fewer than k non-trivial eigenvectors."""
    n = num_nodes
    A = sp.coo_matrix((np.ones(edge_index.size(1)), (edge_index[0].numpy(), edge_index[1].numpy())),
                      shape=(n, n)).tocsr()
    dinv = 1.0 / np.sqrt(np.maximum(np.asarray(A.sum(1)).ravel(), 1.0))
    Ahat = sp.diags(dinv) @ A @ sp.diags(dinv)          # = I - L, same eigenvectors
    m = min(k, n - 1)                                   # non-trivial eigenvectors available
    out = np.zeros((n, k), dtype=np.float32)
    if m > 0:
        if n <= DENSE_LIMIT:
            w, V = np.linalg.eigh(np.eye(n) - Ahat.toarray())           # ascending
        else:                                            # smallest of L = largest of I - L
            v0 = np.random.default_rng(RANDOM_SEED).standard_normal(n)   # fixed ARPACK start
            w, V = sp.linalg.eigsh(Ahat, k=m + 1, which="LA", tol=0, v0=v0)
            w, V = 1.0 - w, V
            order = np.argsort(w)
            w, V = w[order], V[:, order]
        out[:, :m] = V[:, 1:m + 1] * math.sqrt(n)        # skip the trivial eigenvector
    return torch.from_numpy(out)


def hop_distances(edge_index, num_nodes, cap=None):
    """All-pairs hop distance by frontier expansion, as int16 [n, n].

    Unreachable pairs (different connected components) are -1.  With `cap` set the
    expansion stops after `cap` hops and every pair that is further away is reported
    as `cap`; those pairs share the max_dist bias anyway."""
    n = num_nodes
    src, dst = edge_index[0].numpy(), edge_index[1].numpy()
    A = sp.coo_matrix((np.ones(len(src), dtype=np.float32), (src, dst)), shape=(n, n)).tocsr()
    D = np.full((n, n), -1, dtype=np.int16)
    limit = n if cap is None else cap
    if n <= DENSE_LIMIT:
        Ad = np.zeros((n, n), dtype=np.float32)
        Ad[src, dst] = 1.0
        seen = np.eye(n, dtype=bool)
        frontier = seen.copy()
        D[seen] = 0
        for d in range(1, limit + 1):
            frontier = ((frontier.astype(np.float32) @ Ad) > 0) & ~seen
            if not frontier.any():
                break
            D[frontier] = d
            seen |= frontier
    else:
        for s0 in range(0, n, 1024):                     # blocks of sources, columns of F
            s1 = min(s0 + 1024, n)
            F = np.zeros((n, s1 - s0), dtype=np.float32)
            F[np.arange(s0, s1), np.arange(s1 - s0)] = 1.0
            seen = F.astype(bool)
            D[np.arange(s0, s1), np.arange(s0, s1)] = 0
            for d in range(1, limit + 1):
                F = ((A @ F) > 0) & ~seen
                if not F.any():
                    break
                D[s0:s1][F.T] = d
                seen |= F
                F = F.astype(np.float32)
```

```
    if cap is not None:                                  # reached later than the cap, not unreachable
        comp = sp.csgraph.connected_components(A, directed=False)[1]
        far = (D < 0) & (comp[:, None] == comp[None, :])
        D[far] = cap
    return torch.from_numpy(D)
```

notebook cell 13 — Q1

```
class LapPETransformer(GraphTransformer):
    """Absolute encoding: Laplacian eigenvectors, added to the input once."""

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Linear map from the LAP_K eigenvector entries of a node to a node vector of size dim.
        self.lap_proj = nn.Linear(LAP_K, self.dim)

    @staticmethod
    def precompute(edge_index, num_nodes):
        return {"lap": laplacian_eigvecs(edge_index, num_nodes)}

    def inject_absolute(self, h, batch):
        lap = batch["lap"]
        if self.training:                                # eigenvectors are defined up to sign:
            sign = torch.randint(0, 2, (batch["num_graphs"], lap.size(1)),  # flip at random while
                                 device=lap.device, dtype=lap.dtype) * 2 - 1  # training, identity
            lap = lap * sign[batch["batch"]]                                 # at evaluation
        return h + self.lap_proj(lap)


class SPDAttention(Attention):
    """Relative encoding: shortest-path distance, one learnable bias per head."""

    def __init__(self, dim, heads, max_dist=SPD_MAX_DIST):
        super().__init__(dim, heads)
        self.max_dist = max_dist
        # one bias per head for each distance 0..max_dist, plus one for unreachable pairs
        self.spd_table = nn.Embedding(max_dist + 2, heads)

    @staticmethod
    def precompute(edge_index, num_nodes):
        cap = None if num_nodes <= DENSE_LIMIT else SPD_MAX_DIST
        return {"spd": hop_distances(edge_index, num_nodes, cap=cap)}

    def bucket(self, d):
        """Hop distance -> row of spd_table: 0..max_dist, and max_dist + 1 if unreachable."""
        d = d.long()
        return torch.where(d < 0, torch.full_like(d, self.max_dist + 1), d.clamp(max=self.max_dist))

    def forward(self, h, enc, mask):
        B, N, D = h.shape
        q, k, v = self.qkv(h).view(B, N, 3, self.heads, D // self.heads).permute(2, 0, 3, 1, 4)
        scores = q @ k.transpose(-1, -2) / math.sqrt(D // self.heads)      # [B, H, N, N]
        bias = self.spd_table(self.bucket(enc["spd"]))                     # [B, N, N, H]
        scores = scores + bias.permute(0, 3, 1, 2)
        scores = scores.masked_fill(~mask[:, None, None, :], float("-inf"))
        weights = torch.softmax(scores, dim=-1).nan_to_num(0.0)
        return self.out((weights @ v).transpose(1, 2).reshape(B, N, D))
```

notebook cell 16 — Q2

```
EXPANDER_CYCLES = 2       # union of 2 random Hamiltonian cycles = a 4-regular expander


def pair_set(edge_index, num_nodes, n_cycles=EXPANDER_CYCLES, virtual=True, extra=()):
```

```
    """The pair set of one graph: self loops, the graph's edges, an expander built as the
    union of n_cycles random Hamiltonian cycles, and one global virtual node (id num_nodes).
    The cycles are drawn from a generator seeded by the graph itself, so they are fixed."""
    n, node = num_nodes, torch.arange(num_nodes)
    gen = torch.Generator().manual_seed((RANDOM_SEED * 1000003 + n * 7919
                                         + int(edge_index.sum())) % (2 ** 31))
    parts = [torch.stack([node, node]), edge_index, *extra]        # self loops + graph edges
    for _ in range(n_cycles if n >= 3 else 0):                     # expander edges
        p = torch.randperm(n, generator=gen)
        q = torch.roll(p, -1)
        parts += [torch.stack([p, q]), torch.stack([q, p])]
    if virtual:                                                    # global node, both directions
        virt = torch.full_like(node, n)
        parts += [torch.stack([node, virt]), torch.stack([virt, node]), torch.tensor([[n], [n]])]
    return coalesce(torch.cat(parts, 1), num_nodes=n + 1)


class SparseSPDAttention(SPDAttention):
    """Your Q1 attention, restricted to a chosen set of node pairs (Exphormer-style:
    graph neighbourhood + expander + one global virtual node)."""

    def __init__(self, dim, heads, max_dist=SPD_MAX_DIST):
        super().__init__(dim, heads, max_dist)
        # Exphormer gives each kind of attention edge its own bias: local, expander, global
        self.type_table = nn.Embedding(len(PAIR_TYPES), heads)

    @staticmethod
    def attention_pairs(edge_index, num_nodes):
        return pair_set(edge_index, num_nodes, n_cycles=EXPANDER_CYCLES, virtual=True)

    def forward(self, h, enc, pairs):
        M, D = h.shape
        H, dh = self.heads, D // self.heads
        src, dst = pairs                                           # node dst attends to node src
        q, k, v = self.qkv(h).view(M, 3, H, dh).permute(1, 0, 2, 3)
        score = (q[dst] * k[src]).sum(-1) / math.sqrt(dh)          # [P, H]
        virtual = enc["pair_type"] == PAIR_TYPES.index("virtual")  # their spd arrives as 0,
        d = torch.where(virtual, torch.full_like(enc["spd"], -1), enc["spd"])  # but no distance
        score = score + self.spd_table(self.bucket(d)) + self.type_table(enc["pair_type"])
        alpha = segment_softmax(score, dst, num_nodes=M)           # softmax over each node's pairs
        out = torch.zeros(M, H, dh, device=h.device, dtype=h.dtype)
        out.index_add_(0, dst, alpha.unsqueeze(-1) * v[src])       # [M, H, dh]
        return self.out(out.reshape(M, D))
```